

Amanah Care, explained in simple English

MuslimHacks 2026 · Elderly Care track · Concordia University, Montreal · 5 to 6 September 2026 · Team: Abdul Quayum Adekunle, safio

1. What it is, in three sentences

Amanah Care is a private record for a family looking after an elderly parent. The family writes down the elder's routine, logs what was done, and hands the day over to the next person with a clear card: what happened, what is next. Everything is scrambled on the phone before it is sent, so only the family can read it. Not the server, not us.

"Amanah" means a trust you are responsible for. Caring for a parent is exactly that.

2. The problem, in plain words

Ammi is 78. She speaks Urdu, eats halal and soft food, and her prayer times matter to her. Three of her children share her care. Today the whole system is a WhatsApp group. "Did she take the morning ones?" "Who has her this afternoon?" "What did Dr. Rahman say on Wednesday?" "Scroll up."

A chat is a conversation. Care needs a record. Without one, four things go wrong:

- **No plan.** Things get missed, or done twice.
- **No handoff.** The next person does not know what happened.
- **No record of who did what.** One sibling ends up carrying everything, and nobody sees it until they burn out.
- **No privacy.** The whole history of her health lives with a chat company, and with anyone a message was forwarded to.

Evidence. 1 in 4 American Muslims care for an older adult, versus 16% of the general public (ISPU, *Support and Sacrifice*, 2026). 52% of Muslims aged 30 to 49 care for an elder (ISPU, *Sandwiched*). In Canada, 42% of people 15 and over gave unpaid care in 2022, 13.4 million people, and 1.8 million of them cared for both children and adults (Statistics Canada, Canadian Social Survey 2022). Among family caregivers of people with dementia, about 31% have depression and 49% report burden (Collins & Kishita, 2020, meta-analysis). 18% of Muslim caregivers report difficulty finding culturally appropriate care, versus 7% (ISPU 2026, via the MuslimHacks guide). Health

information needs consent and privacy protection under Quebec Law 25, Ontario PHIPA and PIPEDA.

3. What the app does

Routine

The family enters Ammi's plan once: morning meds at 8, breakfast at 8:30, Dhuhr at 1, walk after Asr, night meds at 9 (Fatima), pharmacy pickup on Sundays at 11 (Abdullah), Dr. Rahman on Wednesdays (Fatima). Each item has a category, a time, which days, and who does it.

Today's plan

Every day the routine becomes a checklist. Tap "Done" and it is logged with your name and the time. Items that are past their time and not logged say "not yet". No alarms, no judgment, just what is done and what is open.

Log

Anything else, in two taps: pick a category (meds, meal, prayer, mobility, mood, appointment, transport, note), then what happened ("ate half", "refused food", "walked", "tired"). Add a note if you want.

Hand off

When you leave, you pick who takes over. "What happened" is written for you from your log. "What is next" is written for you from the routine: everything still open today, in order, with who is on it. You can edit both. You send. Their phone shows the card within seconds, with Ammi's preferences on top (language, diet, prayer, modesty). They tap "Accept". You see "accepted" on your side.

Home

The dashboard. Who has Ammi right now. Today's plan, 8 of 12 done. Meds 2 of 3, meals 2 of 3, prayers 3 of 5. Her mood and mobility, last logged, by whom, when. Handoffs waiting. Next pickup and next appointment, with who is on it. Your own part today. The last 7 days. Who carried the week.

Record

Everything logged in the last 30 days, day by day, newest first, with a filter per category. This is the "track of everything" that WhatsApp never gives you.

Log in

The first time, you join with the QR code or the invite code, which proves you were given the family key in person. Then you set a login (your email, or a simple name chosen for

someone without one) and a password. From then on you log in with those. Today the default password is 333.

Ammi's own screen

When the elder logs in, the app looks different: four tabs only, big type, nothing to operate. Who is looking after her today and who comes next. What is coming up. Her prayers today with ticks. What is done. The next visit and pickup. What her family knows about her. Who can read her record.

Privacy

When the family is created, the phone makes a secret key. It never leaves the phone. Other family members get it by scanning a QR code in person. Every note, name, summary and preference is scrambled with that key before it is sent. The server stores the scrambled text and only enough to route it: what type of event, which category, which member id, when.

4. A day with Ammi's family (the demo)

1. Sister A opens Home. It says "Sister A has Ammi since Friday 12:04", 8 of 12 done, next pickup Sunday 11:00, Abdullah.
2. She taps Done on dinner. The plan and the tiles update.
3. She opens Hand off, picks Fatima. What happened and what is next are already filled in. She sends. An animation shows the card travelling: readable on her phone, scrambled on the server, readable again on Fatima's phone.
4. Fatima's phone shows the card, with Ammi's preferences on top. She accepts.
5. Home now says "Fatima has Ammi". Ammi's tab, in big type, says the same. The week's load bar moves.
6. We open the database. Every row is scrambled text. We, the operators, cannot read a word of it.

5. How the privacy works, in plain words

- The key is 256 random bits made by the browser (WebCrypto, AES-GCM). It is never sent to the server.
- The server keeps a fingerprint of the key (a SHA-256 hash). That is enough to check you are family when you join, but it cannot be used to read anything.
- Each message is scrambled with a fresh random number (a 96-bit IV), so two identical notes look different on the wire, and any tampering makes the message unreadable rather than wrong.

- Members join by QR code, face to face, once. For later logins the phone makes a copy of the key locked with the password (PBKDF2 and AES-GCM) and the server keeps that locked copy plus a script hash of the password. The server can check the password. It cannot open the copy.
- The app never offers "share to WhatsApp", because that is exactly where the key must not go.
- The server can count things it can see (how many events per person, per day) without reading any content. That is how the workload bar works.
- There is no AI model anywhere in the loop. The server could not run one on this data even if we wanted it to.

6. How it is built

Every action is an event: "care logged", "handoff opened", "handoff acknowledged", "preferences set", "routine set". Events go into an append-only log (Redis Streams). A small program called the projector copies each event into a database (Postgres) and keeps two simple views current: the list of handoffs, and the counts per person. Phones read those views every few seconds and decrypt what belongs to them. The dashboard is calculated on the phone.

Because everything is an event, we added the whole dashboard, the routine and the record without changing how data is written. They are new ways of reading the same log. Replaying the log rebuilds everything.

It runs with one command (Docker Compose: web, api, projector, redis, postgres). As a fallback, one node command runs the same api and projector with an in-memory database, so the demo does not depend on Docker being alive.

7. What is real, what is simplified, what is not built

REAL AND WORKING	SIMPLIFIED ON PURPOSE	NOT BUILT YET
Create a family, join by QR or code, then log in with email or login + password	Notifications by polling every 4 seconds, not push	Elder consent filter (needs a separate key for support workers)
Elder view in big type	Login exists, but the demo default password is 333 with no rate limiting or recovery yet	Key rotation after a lost phone
Log, routine, today's plan	One Redis stream for all families	Reminders on open handoffs
Handoff: compose, send, accept	Support workers hold the same key as family, so hiding categories from them is only visual	Urdu and Arabic for the elder view
Dashboard, record, workload		Native mobile apps
End-to-end encryption, ciphertext-only server		
Redis Streams → projector → Postgres		

We chose one workflow, the handoff, and built it properly. The right-hand column is designed and on the roadmap, not hidden.

8. What we tested

40 automated tests run the real API routes in-process against the real database schema (loaded into an in-memory Postgres) and push every write through the real projector before checking the results. Coverage on the api, the projector and the client crypto: 100% of lines, 90% of branches, 100% of functions. One test writes a private note ("Ammi refused breakfast, said she felt dizzy"), then checks that the sentence appears in no database row, no stream entry and no API response, and that it decrypts correctly with the family key and not with any other key. `npm test` runs in about 2.5 seconds with no Docker.

9. What comes next

- **Monday:** a one-month pilot with five Montreal families, Urdu and Arabic speaking. Urdu and Arabic for the elder's screen on the tablet on the kitchen table.
- **Next month:** a separate key for support workers so the consent filter is real, not visual. Push notifications. Key rotation. Password policy and recovery.
- **Later:** native apps, offline first, per-family streams.

10. Where AI fits

Only on the phone, where the readable data already is. Never on the server. Three uses, all opt in per family:

- **Say it, do not type it.** "Ammi ate half her lunch, took her meds, seemed tired" becomes three log items. The log already accepts structured items, so a parser plugs in front of it.
- **Read it her way.** Handoff summaries in plain sentences, and in Urdu or Arabic for the elder's view.
- **Patterns, not predictions.** "Refused lunch 3 of the last 5 days." A count shown as a fact. The family decides what it means. No diagnosis, no triage.

Chrome now ships on-device AI (Prompt, Summarizer and Translator APIs on Gemini Nano, stable on desktop), and phones have on-device models. That is how we add AI without breaking "the server cannot read it". We did not ship it in 24 hours because calling a cloud model from the demo would contradict that claim on stage.

11. How to run the demo

```
cd app
docker compose up -d --build          # web :8080, api :4000, projector,
redis, postgres
cd seed && npm install && API=http://localhost:4000 node seed.js  # prints
the join codes

# fallback, no Docker: same api and projector, in-memory database
cd app && npm install && node dev/stack.mjs
```

Open <http://localhost:8080> in one window and <http://127.0.0.1:8080> in another (two origins, two separate sessions). Paste the Sister A code into one Join box and the Fatima code into the other, or log in as *sistera*, *abdullah*, *fatima* or *ammi* with password 333. Each tab is its own session, so three tabs can be three people. Tests: `cd app && npm test`, coverage: `npm run coverage`.

12. Judge questions, with our answers

Technical implementation

What is the delivery format?

A web app, phone first, any browser. Two windows on stage stand in for two phones. Native mobile is roadmap.

What technologies did you use?

Node and Express for the API, Redis Streams for the event log, Postgres for the read models, plain JavaScript with WebCrypto (AES-GCM) in the browser, nginx, Docker Compose. Node's built-in test runner. A vendored QR library. No frameworks and no CDNs, so the demo needs no internet.

How does it work behind the scenes?

The phone encrypts, then POSTs an event. The API checks membership and appends it to the stream. The projector copies it into Postgres and updates the handoff and workload views. Phones poll the views and decrypt. The dashboard is computed on the phone.

Did you build this from scratch during the hackathon?

The spec, the requirement list and two feasibility audits were prepared ahead. All code was written during the hackathon window. QR generation uses a vendored library.

What technical tradeoffs did you make?

Polling instead of push, a demo default password instead of a real credential policy, one stream instead of one per family, and the consent filter deferred. Each is on the honesty slide with the reason.

Impact and users

Who is your target user?

Adult children sharing the care of an elderly parent. The elder is the subject and the owner of the preferences, not the app user. Support workers join with their own role.

How would users discover or adopt it?

Family to family, by QR, in person, by design. Community care programs and masjid welfare committees as the channel. Agencies bring their families.

How many people could benefit?

1 in 4 American Muslims already care for an older adult. Muslims are 8.7% of Greater Montreal. We start with Montreal families, then any family with a shared care load.

What impact could it have?

Fewer missed meds and appointments, a fairer split of the load, less of the strain the research documents, and a health record the family owns.

How would you measure success?

Plan adherence per day, time to accept a handoff, active caregivers per family, and weekly return rate.

Demo and functionality

Is the demo fully functional or partly mocked?

The core loop is real end to end: encrypt, stream, project, store, decrypt. Login is real, with a demo default password. Notifications are polling. The consent filter is not built. Nothing on stage is faked.

Most important feature?

The handoff composed from the record: what happened from the log, what is next from the routine, the elder's preferences on top.

Hardest part?

Keeping the server useful while it can read nothing: deciding which fields stay clear for routing and counting, and building a whole dashboard from decryption on the phone.

Completed versus prototyped?

See section 7.

Business and scalability

Business model?

Free for families. Paid per elder per month by home-care agencies and community programs that need handoff logs and scoped access.

How would it scale?

Per-family streams, push, scoped keys, password policy, offline. Ciphertext-only storage keeps compliance and hosting light.

What would it take to make it a real product?

Password policy and key recovery, native apps, scoped support keys, push, offline, accessibility, Urdu and Arabic.

Who would pay?

Agencies and community organisations. Families optionally for extras such as key escrow between two relatives.

Team and process

How did you divide the work?

Abdul: architecture, event model, crypto, api and projector, dashboard, tests. safio: in-app invite flow, interface cleanup, and the flow view that traces a write hop by hop.

What did you learn?

Cut scope early and say so. One workflow done well beats five half done. Privacy shapes every feature, including the ones you cannot build yet.

What would you do differently?

Tests from hour one. Push notifications budgeted from the start.

How did you prioritise?

A requirement register of 46 items, two audit rounds on what fits in 24 hours, then "cut core": the handoff loop first, everything else labelled roadmap.

Data and privacy

What data does it use?

Care events, handoff summaries, preferences, the routine and member names, all encrypted. In the clear: event type, category, member id and timestamps.

How do you handle privacy?

Family key made in the browser and never sent. AES-GCM per message with a fresh IV. A SHA-256 check of the key on the server. QR in person. Ciphertext-only storage. No third party.

Ethical risks?

Support workers currently hold the same key, so hiding categories from them is visual only. The password-locked copy of the key is only as strong as the password, and the demo default is 333. A lost key means lost data until rotation ships. No diagnosis, dosage or triage anywhere, and the metrics are counts, never judgments.

Does it depend on an LLM?

No. By design the server cannot run one on this data.

Future plans and closing

What would you build next?

Urdu and Arabic for the elder view, a scoped support key, push, key rotation, password policy and recovery.

Will you continue?

Yes. A five-family pilot in Montreal next week.

First step after today?

Publish the repo, deploy on one small VM, recruit the pilot families through community care contacts.

What makes it different?

Task apps organise chores, medication apps remind, marketplaces find carers. None connect plan, actuals, handoff and load with the elder's preferences, and none let only the family read the data.

What help do you need?

Introductions to community care programs and agencies, pilot families, and a designer for the elder view.

What should we remember?

The server cannot read it. Open the database and see.

Why should this win?

It solves the coordination problem the track asked for, with the elder's preferences built in, privacy that is cryptographic rather than a promise, an honest scope, and a test suite behind it.

13. Sources

ISPU, Support and Sacrifice: Muslims Who Care for Older Adults Face Emotional Strain, 2026, from American Muslim Poll 2025 (ispu.org/reports-and-analysis/support-and-sacrifice/). ISPU, Sandwiched: Muslims Are Most Likely to Be Caring for Aging Parents and Have School-Aged Children (ispu.org/muslim-families/). Statistics Canada, The Daily, 8 November 2022, Canadian Social Survey 2022; and 2 April 2024, "Sandwiched" between multiple unpaid caregiving responsibilities. Collins, R. N. and Kishita, N., Prevalence of depression and burden among informal care-givers of people with dementia: a meta-analysis, Ageing & Society, 2020. MuslimHacks 2026 Challenge Guide, Elderly Care, research and fact sheets by Process Plus. Statistics Canada, Census 2021, religion release. Chrome for Developers, Built-in AI (developer.chrome.com/docs/ai/built-in).